

Stock Monitoring and Alerts - Low-Level Design

Version: 1.0.0 | Date: 2026-05-27 | Status: Draft

Level: Enterprise | Source HLD:
`docs/02-design/features/stock-monitoring-alerts-hld.design.md`

1. LLD Scope

This Low-Level Design defines concrete implementation contracts for the Stock Monitoring and Alerts platform. It is based on the approved parent plan and split HLD for stock-monitoring-alerts.

This document focuses on implementation detail:

- REST APIs and payload contracts.
- Database schema placeholders.
- Domain models and validation rules.
- OIC switch branches and integration steps.
- API Gateway route, header, and auth mapping.
- Python engine module design.
- Provider abstraction and mock/live provider behavior.
- VBS data-binding assumptions.
- Test, smoke, and security cases.

The HLD remains the source for architecture context, deployment topology, and layer-level responsibilities.

2. Design Constraints

- Python engine must remain private/internal.
- Public access must go through OCI API Gateway over HTTPS on port 443.
- OIC must orchestrate workflow and notification branches.
- Database persistence is required from the start.
- Mock provider is default for local development and CI.
- Live provider support must use environment/configuration.
- Secrets must never be committed.

3. API Conventions

3.1 Route Prefixes

Route Family	Prefix	Caller	Purpose
Public dashboard API	/api/v1	VBS through OCI API Gateway	Browser-facing contract
Internal engine API	/internal/v1	OIC/private integration path	Private Python engine contract

3.2 Payload Rules

- JSON only for request and response bodies.
- All timestamps use ISO 8601 UTC.

- Price and threshold values use decimal strings in API payloads.
- Request payloads use camelCase.
- Database fields use snake_case.
- All responses include a meta.correlationId.
- Batch operations may return partial success with item-level errors.

3.3 Header and Correlation ID Rules

Header	Direction	Required	Description
Authorization	VBS to API Gateway	Yes in cloud	Bearer/session token validated by gateway
X-Correlation-Id	All layers	Recommended	Generated at gateway when absent
X-Authenticated-User-Id	Gateway/OIC to engine	Yes for user APIs	Trusted user reference
X-Authenticated-User-Email	Gateway/OIC to engine	Optional	User email for display/audit
X-Auth-Scopes	Gateway/OIC to engine	Optional	Authorization scopes/roles

Correlation behavior:

- API Gateway accepts an inbound correlation ID only when policy allows it.
- API Gateway generates a correlation ID if absent.
- OIC passes the same correlation ID to the Python engine.
- Python persists correlation ID on execution logs and notification audit rows.
- Error responses preserve the same correlation ID.

4. Common JSON Envelopes

4.1 Common Response Envelope

```
{
  "data": {},
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

4.2 Common Error Envelope

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid.",
    "details": [
      {
        "field": "symbol",
        "reason": "Symbol is required."
      }
    ]
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

```
}
```

5. REST API Endpoint Specifications

5.1 Health and Readiness

GET /internal/v1/health

Purpose: process liveness.

Response:

```
{
  "data": {
    "status": "ok",
    "service": "stock-monitoring-engine"
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

GET /internal/v1/ready

Purpose: dependency readiness.

Response:

```
{
  "data": {
    "status": "ready",
    "database": "ok",
    "marketDataProvider": "mock"
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

5.2 Watchlists

GET /api/v1/watchlists

Gateway route to OIC, then engine GET /internal/v1/watchlists.

Query parameters:

Name	Required	Description
enabled	No	Optional boolean filter
limit	No	Default 100, max 500
cursor	No	Pagination cursor

Response:

```
{
  "data": {
    "items": [
      {
        "watchlistId": "wl_001",
        "symbol": "ORCL",
        "displayName": "Oracle Corporation",
        "enabled": true,
        "createdAt": "2026-05-27T01:00:00Z",
        "updatedAt": "2026-05-27T01:00:00Z"
      }
    ],
    "nextCursor": null
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

POST /api/v1/watchlists

Gateway route to OIC, then engine POST /internal/v1/watchlists.

Request:

```
{
  "symbol": "ORCL",
  "displayName": "Oracle Corporation",
  "enabled": true
}
```

Response:

```
{
  "data": {
    "watchlistId": "wl_001",
    "symbol": "ORCL",
    "displayName": "Oracle Corporation",
    "enabled": true,
    "createdAt": "2026-05-27T01:00:00Z",
    "updatedAt": "2026-05-27T01:00:00Z"
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

PATCH /api/v1/watchlists/{watchlistId}

Request:

```
{
  "displayName": "Oracle",
  "enabled": false
}
```

Response:

```
{
  "data": {
    "watchlistId": "wl_001",
    "symbol": "ORCL",
    "displayName": "Oracle",
    "enabled": false,
    "updatedAt": "2026-05-27T01:05:00Z"
  },
  "meta": {
```

```

    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:05:00Z"
  }
}

```

DELETE /api/v1/watchlists/{watchlistId}

Behavior: soft delete by setting deleted_at; historical quote and alert records remain.

Response:

```

{
  "data": {
    "watchlistId": "wl_001",
    "deleted": true
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:06:00Z"
  }
}

```

5.3 Alert Targets

GET /api/v1/alerts

Query parameters:

Name	Required	Description
watchlistId	No	Filter alerts by watchlist
symbol	No	Filter alerts by symbol
enabled	No	Filter enabled/disabled alerts
limit	No	Default 100, max 500

Response:

```

{
  "data": {
    "items": [
      {
        "alertId": "al_001",
        "watchlistId": "wl_001",
        "symbol": "ORCL",
        "ruleType": "price_threshold",
        "operator": "gte",
        "threshold": "140.00",
        "currency": "USD",
        "enabled": true,
        "notificationChannel": "email",
        "lastEvaluationState": "not_triggered",
        "createdAt": "2026-05-27T01:00:00Z",
        "updatedAt": "2026-05-27T01:00:00Z"
      }
    ]
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}

```

POST /api/v1/alerts

Request:

```
{
  "watchlistId": "wl_001",
  "ruleType": "price_threshold",
  "operator": "gte",
  "threshold": "140.00",
  "currency": "USD",
  "enabled": true,
  "notificationChannel": "email"
}
```

Response:

```
{
  "data": {
    "alertId": "al_001",
    "watchlistId": "wl_001",
    "ruleType": "price_threshold",
    "operator": "gte",
    "threshold": "140.00",
    "currency": "USD",
    "enabled": true,
    "notificationChannel": "email",
    "lastEvaluationState": "unknown"
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

PATCH /api/v1/alerts/{alertId}

Request:

```
{
  "threshold": "145.00",
  "enabled": true,
  "notificationChannel": "slack"
}
```

Behavior:

- Update mutable fields.
- Write target history when threshold, enabled state, channel, or operator changes.
- Reset duplicate suppression state if the threshold/operator changes.

DELETE /api/v1/alerts/{alertId}

Behavior: soft delete or disable the alert target while retaining history.

5.4 Quotes

POST /api/v1/quotes/refresh

Gateway route to OIC, then engine POST /internal/v1/quotes/refresh.

Request:

```
{
  "symbols": ["ORCL", "AAPL"]
}
```

```
}
```

Response:

```
{
  "data": {
    "provider": "mock",
    "quotes": [
      {
        "quoteSnapshotId": "qs_001",
        "symbol": "ORCL",
        "price": "139.42",
        "currency": "USD",
        "quoteTime": "2026-05-27T01:00:00Z",
        "provider": "mock"
      }
    ],
    "errors": []
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

GET /api/v1/quotes?symbol=ORCL&limit=100

Purpose: return persisted quote snapshots for dashboard graphing.

Response:

```
{
  "data": {
    "items": [
      {
        "quoteSnapshotId": "qs_001",
        "symbol": "ORCL",
        "price": "139.42",
        "currency": "USD",
        "quoteTime": "2026-05-27T01:00:00Z",
        "provider": "mock"
      }
    ]
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

5.5 Alert Evaluation

POST /api/v1/alerts/evaluate

Gateway route to OIC, then engine POST /internal/v1/alerts/evaluate.

Request:

```
{
  "symbols": ["ORCL"],
  "dryRun": false,
  "triggerSource": "dashboard"
}
```

Response:


```
{
  "data": {
    "executionId": "ex_001",
    "provider": "mock",
    "evaluatedCount": 2,
    "triggeredCount": 1,
    "notifications": [
      {
        "notificationAuditId": "nt_001",
        "alertId": "al_001",
        "symbol": "ORCL",
        "channel": "email",
        "status": "pending",
        "reason": "price 141.10 gte threshold 140.00"
      }
    ],
    "errors": []
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:00:00Z"
  }
}
```

GET /api/v1/alerts/history?symbol=ORCL&limit=50

Purpose: return target changes and evaluation summaries for dashboard audit views.

5.6 Notifications

GET /api/v1/notifications?status=pending&limit=50

Purpose: return user-visible notification audit records.

PATCH /internal/v1/notifications/{notificationAuditId}/status

Caller: OIC notification dispatch flow only.

Request:

```
{
  "status": "sent",
  "providerMessageId": "email-123",
  "failureReason": null,
  "deliveredAt": "2026-05-27T01:01:00Z"
}
```

Response:

```
{
  "data": {
    "notificationAuditId": "nt_001",
    "status": "sent",
    "updatedAt": "2026-05-27T01:01:00Z"
  },
  "meta": {
    "correlationId": "corr-123",
    "timestamp": "2026-05-27T01:01:00Z"
  }
}
```

6. Database Schema Placeholders

6.1 Data Type Assumptions

- IDs are UUID-compatible string values.
- Timestamps are UTC.
- Prices and thresholds use numeric/decimal storage.
- Soft delete uses `deleted_at`.
- Provider-specific details use JSON metadata fields.

6.2 Users

```
CREATE TABLE users (  
  user_id VARCHAR(128) PRIMARY KEY,  
  email VARCHAR(320),  
  display_name VARCHAR(255),  
  external_subject VARCHAR(255),  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL  
);
```

6.3 Watchlist Model

```
CREATE TABLE watchlists (  
  watchlist_id VARCHAR(64) PRIMARY KEY,  
  user_id VARCHAR(128) NOT NULL REFERENCES users(user_id),  
  symbol VARCHAR(16) NOT NULL,  
  display_name VARCHAR(255),  
  enabled BOOLEAN NOT NULL DEFAULT TRUE,  
  deleted_at TIMESTAMP NULL,  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL,  
  UNIQUE (user_id, symbol)  
);  
  
CREATE INDEX idx_watchlists_user_enabled  
  ON watchlists(user_id, enabled);
```

6.4 Alert Rule Model

```
CREATE TABLE alert_targets (  
  alert_id VARCHAR(64) PRIMARY KEY,  
  user_id VARCHAR(128) NOT NULL REFERENCES users(user_id),  
  watchlist_id VARCHAR(64) NOT NULL REFERENCES watchlists(watchlist_id),  
  rule_type VARCHAR(64) NOT NULL,  
  operator VARCHAR(16) NOT NULL,  
  threshold NUMERIC(18, 6) NOT NULL,  
  currency VARCHAR(8) NOT NULL,  
  enabled BOOLEAN NOT NULL DEFAULT TRUE,  
  notification_channel VARCHAR(32) NOT NULL,  
  last_evaluation_state VARCHAR(32) NOT NULL DEFAULT 'unknown',  
  last_triggered_at TIMESTAMP NULL,  
  cooldown_seconds INTEGER NULL,  
  metadata_json TEXT NULL,  
  deleted_at TIMESTAMP NULL,  
  created_at TIMESTAMP NOT NULL,  
  updated_at TIMESTAMP NOT NULL  
);  
  
CREATE INDEX idx_alert_targets_user_enabled  
  ON alert_targets(user_id, enabled);  
  
CREATE INDEX idx_alert_targets_watchlist  
  ON alert_targets(watchlist_id);
```

Allowed alert values:

- rule_type: price_threshold
- operator: gte, lte
- last_evaluation_state: unknown, not_triggered, triggered, suppressed_duplicate, error

6.5 Quote Snapshot Model

```
CREATE TABLE quote_snapshots (
  quote_snapshot_id VARCHAR(64) PRIMARY KEY,
  symbol VARCHAR(16) NOT NULL,
  price NUMERIC(18, 6) NOT NULL,
  currency VARCHAR(8) NOT NULL,
  quote_time TIMESTAMP NOT NULL,
  provider VARCHAR(64) NOT NULL,
  provider_reference VARCHAR(255),
  raw_metadata_json TEXT NULL,
  created_at TIMESTAMP NOT NULL
);

CREATE INDEX idx_quote_snapshots_symbol_time
ON quote_snapshots(symbol, quote_time DESC);
```

6.6 Target History Model

```
CREATE TABLE target_history (
  target_history_id VARCHAR(64) PRIMARY KEY,
  alert_id VARCHAR(64) NOT NULL REFERENCES alert_targets(alert_id),
  user_id VARCHAR(128) NOT NULL REFERENCES users(user_id),
  change_type VARCHAR(32) NOT NULL,
  previous_value_json TEXT NULL,
  new_value_json TEXT NOT NULL,
  changed_by VARCHAR(128) NOT NULL,
  changed_at TIMESTAMP NOT NULL
);

CREATE INDEX idx_target_history_alert_time
ON target_history(alert_id, changed_at DESC);
```

Allowed change_type values:

- created
- threshold_updated
- enabled
- disabled
- channel_updated
- deleted

6.7 Execution Log Model

```
CREATE TABLE execution_logs (
  execution_id VARCHAR(64) PRIMARY KEY,
  user_id VARCHAR(128),
  trigger_source VARCHAR(32) NOT NULL,
  provider VARCHAR(64) NOT NULL,
  status VARCHAR(32) NOT NULL,
  evaluated_count INTEGER NOT NULL DEFAULT 0,
  triggered_count INTEGER NOT NULL DEFAULT 0,
  error_count INTEGER NOT NULL DEFAULT 0,
  correlation_id VARCHAR(128),
  started_at TIMESTAMP NOT NULL,
  completed_at TIMESTAMP NULL,
  details_json TEXT NULL
);
```

```
CREATE INDEX idx_execution_logs_time
ON execution_logs(started_at DESC);
```

6.8 Notification Audit Model

```
CREATE TABLE notification_audit (
  notification_audit_id VARCHAR(64) PRIMARY KEY,
  execution_id VARCHAR(64) REFERENCES execution_logs(execution_id),
  alert_id VARCHAR(64) NOT NULL REFERENCES alert_targets(alert_id),
  user_id VARCHAR(128) NOT NULL REFERENCES users(user_id),
  symbol VARCHAR(16) NOT NULL,
  channel VARCHAR(32) NOT NULL,
  destination_ref VARCHAR(255),
  status VARCHAR(32) NOT NULL,
  reason TEXT NOT NULL,
  provider_message_id VARCHAR(255),
  failure_reason TEXT,
  created_at TIMESTAMP NOT NULL,
  sent_at TIMESTAMP NULL,
  delivered_at TIMESTAMP NULL,
  updated_at TIMESTAMP NOT NULL
);

CREATE INDEX idx_notification_audit_user_status
ON notification_audit(user_id, status);

CREATE INDEX idx_notification_audit_created
ON notification_audit(created_at DESC);
```

Allowed status values:

- pending
- simulated
- sent
- skipped
- failed
- retry_scheduled

7. Mapping Logic

7.1 VBS to API Gateway

VBS Action	Gateway Route	Payload Source
Load watchlists	GET /api/v1/watchlists	Dashboard data provider
Add symbol	POST /api/v1/watchlists	Watchlist form
Toggle symbol	PATCH /api/v1/watchlists/{id}	Toggle state
Create alert	POST /api/v1/alerts	Alert target form
Update alert	PATCH /api/v1/alerts/{id}	Alert target form
Refresh quotes	POST /api/v1/quotes/refresh	Selected symbols
Evaluate alerts	POST /api/v1/alerts/evaluate	Dashboard action/schedule
Load notifications	GET /api/v1/notifications	Notification table

7.2 API Gateway to OIC

Gateway Route	OIC Integration	Notes
GET /api/v1/watchlists	StockDashboardWatchlistFlow	Query user watchlist
POST /api/v1/watchlists	StockDashboardWatchlistFlow	Create symbol entry
PATCH /api/v1/watchlists/{id}	StockDashboardWatchlistFlow	Update enabled/display name
DELETE /api/v1/watchlists/{id}	StockDashboardWatchlistFlow	Soft delete
GET /api/v1/alerts	StockDashboardAlertFlow	Query alert targets
POST /api/v1/alerts	StockDashboardAlertFlow	Create alert target
PATCH /api/v1/alerts/{id}	StockDashboardAlertFlow	Update target
POST /api/v1/quotes/refresh	StockQuoteRefreshFlow	Refresh quotes
POST /api/v1/alerts/evaluate	StockAlertEvaluationFlow	Evaluate alerts
GET /api/v1/notifications	StockNotificationStatusFlow	Query notification audit

7.3 OIC to Python Engine

OIC Action	Engine Endpoint	Mapping
List watchlists	GET /internal/v1/watchlists	Pass trusted user headers
Create watchlist	POST /internal/v1/watchlists	Normalize symbol uppercase
Update watchlist	PATCH /internal/v1/watchlists/{id}	Preserve path ID
Create alert	POST /internal/v1/alerts	Pass alert request body
Update alert	PATCH /internal/v1/alerts/{id}	Preserve path ID
Refresh quotes	POST /internal/v1/quotes/refresh	Pass symbol array
Evaluate alerts	POST /internal/v1/alerts/evaluate	Include trigger source
Update notification	PATCH /internal/v1/notifications/{id}/status	Send dispatch result

7.4 Engine to Database

Engine Service	Database Tables
Watchlist service	users, watchlists
Alert service	alert_targets, target_history
Quote service	quote_snapshots, execution_logs
Evaluation service	alert_targets, quote_snapshots, execution_logs, notification_audit
Notification audit service	notification_audit

8. API Gateway Route, Header, and Auth Mapping

8.1 Route Mapping

Gateway Route	Method	Target
/api/v1/watchlists	GET/POST	OIC watchlist integration
/api/v1/watchlists/{watchlistId}	PATCH/DELETE	OIC watchlist integration
/api/v1/alerts	GET/POST	OIC alert integration

Gateway Route	Method	Target
/api/v1/alerts/{alertId}	PATCH/DELETE	OIC alert integration
/api/v1/quotes/refresh	POST	OIC quote refresh integration
/api/v1/quotes	GET	OIC quote query integration
/api/v1/alerts/evaluate	POST	OIC alert evaluation integration
/api/v1/alerts/history	GET	OIC alert history integration
/api/v1/notifications	GET	OIC notification status integration

8.2 Header Mapping

Source Claim/Header	Gateway Output Header
subject/user id claim	X-Authenticated-User-Id
email claim	X-Authenticated-User-Email
scope/roles claim	X-Auth-Scopes
request ID or generated ID	X-Correlation-Id

8.3 Suggested Authorization Scopes

- stock.watchlist.read
- stock.watchlist.write
- stock.alert.read
- stock.alert.write
- stock.quote.read
- stock.alert.evaluate
- stock.notification.read

9. OIC Switch Branch Structures

9.1 Alert Evaluation Switch

```

Switch: EvaluationResult
  Case triggeredCount > 0
    -> For each notification: invoke NotificationDispatchFlow
    -> Patch notification status from channel result
    -> Return triggered summary
  Case triggeredCount == 0 and errorCount == 0
    -> Return no-trigger summary
  Case errorCount > 0 and evaluatedCount > 0
    -> Return partial-success summary
  Case provider unavailable
    -> Write audit failure branch
    -> Return degraded response
  Otherwise
    -> System error branch

```

9.2 Notification Channel Switch

```

Switch: NotificationChannel
  Case email

```

```

-> Email adapter
Case sms
-> SMS adapter
Case slack
-> Slack adapter
Case push
-> Push adapter
Case simulated
-> Mark simulated
Otherwise
-> Mark skipped with unsupported_channel

```

9.3 Dashboard Command Switch

```

Switch: DashboardCommand
Case create_watchlist
Case update_watchlist
Case delete_watchlist
Case create_alert
Case update_alert
Case delete_alert
Case refresh_quotes
Case evaluate_alerts
Default validation_error

```

10. OIC Integration Flow Steps

10.1 Watchlist Flow

```

Receive request from API Gateway
Validate identity headers
Switch by method/path action
Map request payload to engine payload
Invoke engine private endpoint
Map engine response to common dashboard envelope
Return to API Gateway

```

10.2 Alert Target Flow

```

Receive request
Validate user context and payload
Invoke create/update/delete/list alert endpoint
If threshold changed, engine writes target history
Return normalized alert payload

```

10.3 Quote Refresh Flow

```

Receive symbols
Validate symbol list
Invoke engine quote refresh
Switch on provider errors
Return quotes and partial errors

```

10.4 Alert Evaluation Flow

```

Receive dashboard or schedule trigger
Invoke engine alert evaluation
Switch on triggeredCount/errorCount
For triggered notifications, invoke NotificationDispatchFlow
Patch notification statuses back to engine
Return evaluation summary

```

10.5 Notification Dispatch Flow

```
Receive notification audit item
Switch on channel
Invoke downstream connector
On success, patch notification status sent/simulated
On failure, patch failed or retry_scheduled
Return channel result
```

11. Validation Rules

11.1 Symbol Validation

- Required.
- Trim whitespace.
- Convert to uppercase.
- Length 1 to 16.
- Allowed characters: uppercase letters, numbers, ., -.
- MVP mock provider may reject unknown fixtures with `SYMBOL_NOT_FOUND`.

11.2 Threshold Validation

- Required for threshold alerts.
- Decimal string in API payload.
- Greater than zero.
- Maximum precision: 6 decimal places.
- Currency required.
- Currency is uppercase and length 3 to 8.

11.3 Alert Validation

- `ruleType` must be `price_threshold` for MVP.
- `operator` must be `gte` or `lte`.
- `notificationChannel` must be `email`, `sms`, `slack`, `push`, or `simulated`.
- Disabled watchlist items cannot create enabled alerts unless explicitly re-enabled.
- User must own the referenced watchlist before creating an alert.

11.4 Identity Validation

- User-scoped APIs require `X-Authenticated-User-Id` after gateway/OIC.
- Local MVP may use `LOCAL_DEV_USER_ID`.
- Engine returns `UNAUTHENTICATED` if user context is absent for user routes.

12. Error Handling Rules

12.1 Error Codes

Code	HTTP Status	Description
UNAUTHENTICATED	401	Missing or invalid identity context
FORBIDDEN	403	Authenticated user lacks permission
NOT_FOUND	404	Resource not found or not owned by user
VALIDATION_ERROR	400	Invalid payload or route parameter
SYMBOL_NOT_FOUND	422	Provider cannot resolve symbol
PROVIDER_RATE_LIMITED	429	Live provider rate limit reached
PROVIDER_UNAVAILABLE	503	Provider timeout or outage
DUPLICATE_RESOURCE	409	Duplicate watchlist/alert conflict
SYSTEM_ERROR	500	Unexpected engine failure

12.2 Layer Error Behavior

- API Gateway rejects invalid auth and malformed public requests before OIC.
- OIC maps engine errors into dashboard-friendly envelopes.
- Python engine logs internal context but returns safe error messages.
- Provider failures do not abort an entire batch if other symbols can be evaluated.
- Notification failures update audit status and may schedule retry through OIC.

13. Module-Level Python Engine Design

13.1 Proposed Package Structure

```
stock_monitoring_engine/  
  app.py  
  config.py  
  api/  
    dependencies.py  
    routes_health.py  
    routes_watchlists.py  
    routes_alerts.py  
    routes_quotes.py  
    routes_notifications.py  
    error_handlers.py  
  domain/  
    models.py  
    alert_rules.py  
    errors.py  
  services/  
    watchlist_service.py  
    alert_service.py  
    quote_service.py  
    evaluation_service.py  
    notification_audit_service.py  
    execution_log_service.py  
  providers/  
    base.py  
    mock_provider.py  
    live_provider.py  
    registry.py
```

```

repositories/
  database.py
  users_repository.py
  watchlists_repository.py
  alerts_repository.py
  quotes_repository.py
  history_repository.py
  notifications_repository.py
  executions_repository.py
migrations/
tests/

```

13.2 Module Responsibilities

Module	Responsibility
config.py	Environment loading, provider selection, local mode
api/*	Request parsing, response envelopes, route handlers
domain/alert_rules.py	Pure alert evaluation logic
providers/base.py	Provider protocol/interface
providers/mock_provider.py	Deterministic local/CI quotes
providers/live_provider.py	Future live integration adapter
services/evaluation_service.py	Orchestrates quote fetch, persistence, rule evaluation
repositories/*	Database access and transaction boundaries

14. Alert Rule and Evaluation Logic

14.1 Rule Model

Field	Description
ruleType	MVP value is price_threshold
operator	gte or lte
threshold	Decimal string in API, numeric in database
currency	Currency associated with threshold
enabled	Whether rule participates in evaluation
lastEvaluationState	Duplicate suppression and history state

14.2 Evaluation Algorithm

```

load enabled alert targets
group targets by symbol
fetch latest quotes for symbols
persist quote snapshots
for each alert target:
  compare latest quote price with threshold using operator
  if condition false:
    set state not_triggered
  if condition true and previous state was triggered:
    set state suppressed_duplicate
  if condition true and previous state was not triggered/unknown/error:
    set state triggered
    create notification audit row
write execution log summary
return evaluated count, triggered count, notification audit IDs

```

14.3 Comparison Rules

Operator	Trigger Condition
gte	latest price >= threshold
lte	latest price <= threshold

14.4 Duplicate Suppression

- If previous state is triggered and condition remains true, do not create a new notification.
- If condition becomes false, transition to not_triggered.
- If condition becomes true again after false, create a new notification.
- Future cooldown logic may allow periodic reminders.

15. Provider Abstraction Design

15.1 Provider Interface

```
MarketDataProvider
get_quote(symbol) -> QuoteResult
get_quotes(symbols) -> QuoteBatchResult
health() -> ProviderHealth
```

Provider result fields:

- symbol
- price
- currency
- quoteTime
- provider
- providerReference
- rawMetadata

15.2 Mock Provider Behavior

- Default provider for local development and CI.
- Returns deterministic quotes for fixture symbols.
- Supports scenarios for trigger, no-trigger, unknown symbol, stale data, timeout, and provider unavailable.
- Requires no credentials and no network access.

Suggested mock configuration:

```
MARKET_DATA_PROVIDER=mock
MOCK_PROVIDER_SCENARIO=default
```

15.3 Live Provider Configuration Strategy

Live provider support is environment/configuration driven:

```
MARKET_DATA_PROVIDER=live
MARKET_DATA_API_BASE_URL=<configured>
MARKET_DATA_API_KEY=<secret>
MARKET_DATA_TIMEOUT_SECONDS=10
MARKET_DATA_MAX_RETRIES=3
```

Live provider rules:

- Credentials are injected through environment or secret management.
- Provider-specific errors map to normalized error codes.
- Provider-specific payloads stay inside adapter code.
- Alert evaluation consumes only normalized quote results.

16. VBS Screen and Data-Binding Assumptions

16.1 Screens

Screen	Purpose	Data Sources
Login	User authentication	Identity provider/VBS auth
Dashboard	Summary widgets and quote graph	/quotes, /alerts/history
Watchlist	Add/edit/delete symbols	/watchlists
Alert Targets	Configure thresholds and toggles	/alerts
Alert History	View evaluations and target changes	/alerts/history
Notifications	View delivery/audit status	/notifications

16.2 Data Binding

- Watchlist table binds to `data.items` from GET `/api/v1/watchlists`.
- Alert form binds to selected watchlist row and alert model.
- Graph binds to quote snapshots ordered by `quoteTime`.
- Notification status chips bind to `status`.
- Error banners bind to common error envelope.

16.3 UX Behavior

- Save buttons disable during in-flight API calls.
- Toggle actions update only after API success.
- Field validation messages bind to `error.details`.
- Notification statuses refresh after alert evaluation completes.

17. Configuration and Secret Handling

17.1 Local MVP Configuration

```
APP_ENV=local
DATABASE_URL=sqlite:///data/local/stock_mon.db
MARKET_DATA_PROVIDER=mock
LOCAL_DEV_USER_ID=local-user
MOCK_PROVIDER_SCENARIO=default
LOG_LEVEL=INFO
```

17.2 Secret Rules

- `.env` and `.env.*` stay ignored.
- `.env.example` may be tracked with placeholders.
- Provider API keys are never committed.
- OCI keys, wallets, certificates, and Terraform variable files are never committed.

18. Test Cases

18.1 Unit Tests

- Alert `gte` triggers when price equals threshold.
- Alert `gte` triggers when price is above threshold.
- Alert `gte` does not trigger when price is below threshold.
- Alert `lte` triggers when price equals threshold.
- Alert `lte` triggers when price is below threshold.
- Alert `lte` does not trigger when price is above threshold.
- Duplicate suppression skips notification when previous state is triggered.
- Transition from triggered to not triggered resets duplicate suppression.
- Mock provider returns deterministic quote fixtures.
- Provider error maps to normalized provider error code.

18.2 API Tests

- `GET /internal/v1/health` returns `ok`.
- `GET /internal/v1/ready` returns database and provider readiness.
- `POST /internal/v1/watchlists` creates uppercase symbol.
- Duplicate watchlist symbol returns `DUPLICATE_RESOURCE`.
- Missing user header returns `UNAUTHENTICATED`.
- Invalid threshold returns `VALIDATION_ERROR`.
- `POST /internal/v1/alerts/evaluate` creates notification audit for triggered alert.
- `PATCH /internal/v1/notifications/{id}/status` updates notification audit status.

18.3 Integration Tests

- Seed user, watchlist, and alert target.
- Run mock quote refresh.
- Run alert evaluation.
- Verify quote snapshot row exists.
- Verify execution log row exists.
- Verify notification audit row exists when threshold triggers.
- Verify no duplicate notification when condition remains true.

19. Smoke Tests

19.1 Local MVP Smoke Sequence

```
start engine
GET /internal/v1/health
GET /internal/v1/ready
POST /internal/v1/watchlists
POST /internal/v1/alerts
POST /internal/v1/quotes/refresh
POST /internal/v1/alerts/evaluate
GET /internal/v1/notifications
```

19.2 Cloud Smoke Sequence

```
VBS or test client calls API Gateway /api/v1/watchlists
API Gateway validates auth and routes to OIC
OIC calls private engine
Engine persists response
OIC returns mapped payload
API Gateway returns HTTPS response
Alert evaluation creates OIC notification branch
Notification audit status is patched back to engine
```

20. Security Tests

- Direct public access to Python engine endpoint fails.
- API Gateway route without auth fails.
- Engine user route without trusted identity header fails.
- User cannot read another user's watchlist or alerts.
- .env, provider keys, wallets, certificates, and Terraform variable files remain untracked.
- OIC notification status patch endpoint rejects calls without trusted integration context.

21. LLD Acceptance Checklist

- [x] REST API endpoint specifications are documented.
- [x] JSON request and response payloads are documented.
- [x] Common response envelope is documented.
- [x] Common error envelope is documented.

- [x] Header and correlation ID rules are documented.
- [x] Database schema placeholders are documented.
- [x] Watchlist model is documented.
- [x] Alert rule model is documented.
- [x] Quote snapshot model is documented.
- [x] Target history model is documented.
- [x] Execution log model is documented.
- [x] Notification audit model is documented.
- [x] OIC Switch branch structures are documented.
- [x] API Gateway route/header/auth mapping is documented.
- [x] OIC integration flow steps are documented.
- [x] Mapping logic between VBS, API Gateway, OIC, Python engine, and database is documented.
- [x] Validation rules are documented.
- [x] Error handling rules are documented.
- [x] Module-level Python engine design is documented.
- [x] Provider abstraction design is documented.
- [x] Mock provider behavior is documented.
- [x] Live provider configuration strategy is documented.
- [x] VBS screen/data-binding assumptions are documented.
- [x] Test cases are documented.
- [x] Smoke tests are documented.
- [x] Security tests are documented.
- [x] Document stays LLD-only with concrete implementation detail.